`Terraform-04`

1). Watch **Terraform-04** video.
2). Execute the script shown in the video.

Task 2A — Provider Version Constraints

Step 1 — Create a project folder

Open VS Code → Terminal → New Terminal (PowerShell).

Execute:

```
cd $HOME
mkdir terraform-04
cd terraform-04
mkdir task-2a-provider
cd task-2a-provider
code .
```

Step 2 — Create `main.tf`

In VS Code, create a file named `main.tf` and paste:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "2.4.0"
    }
  }
}

resource "local_file" "my_file" {
  filename = "${path.module}/pets.txt"
  content  = "Welcome to Terraform-04"
}
```

What we're learning:

- `required_providers` — Specifies the Terraform providers needed.
- `source` — Identifies where the provider comes from.
- `version` — Restricts the provider to version `2.4.0`.
- `local_file` — Creates a file on your computer.

Step 3 — Execute Terraform commands

Run these one by one:

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

When Terraform asks for confirmation, type `yes`.

Step 4 — Verify the output

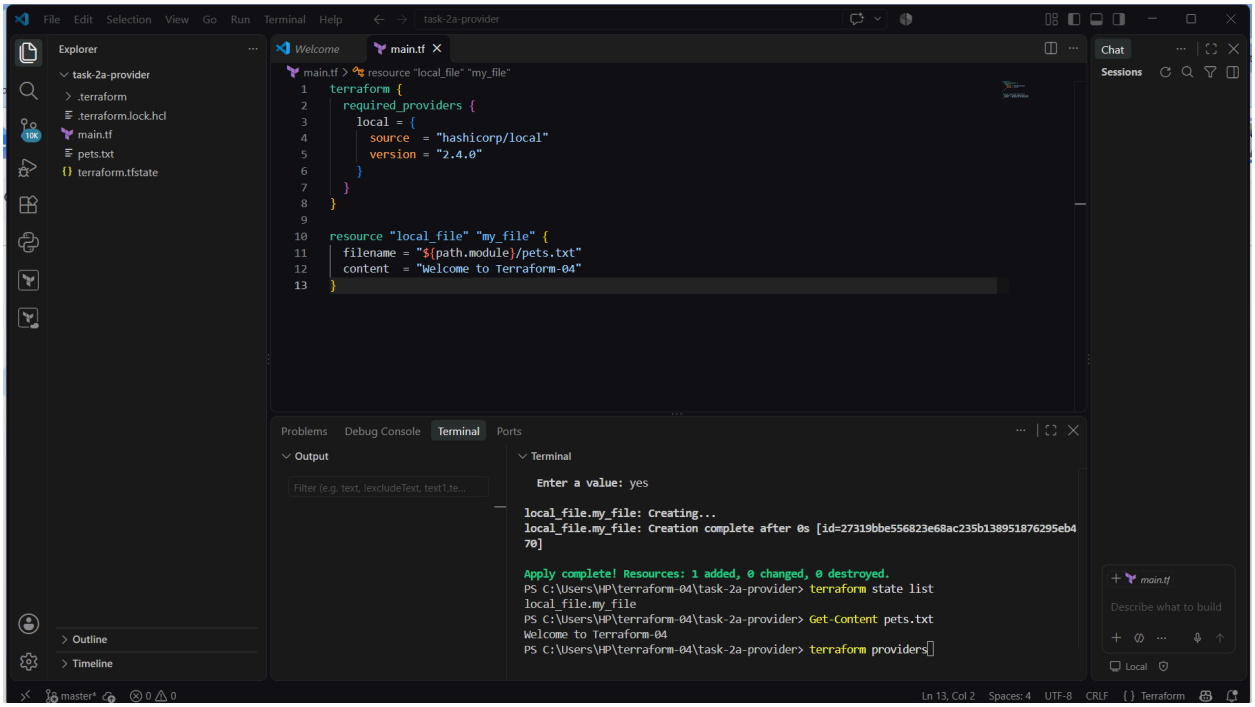
terraform state list
Get-Content pets.txt
terraform providers

Expected file content:

Welcome to Terraform-04

Expected state entry:

local_file.my_file



Task 2B — Terraform Data Sources

1. What is a Data Source?

A Terraform Data Source reads information that already exists, instead of creating that infrastructure.

Simple example: You manually create `dogs.txt`. Terraform reads its content and uses that content to create another file named `pets.txt`.

dogs.txt

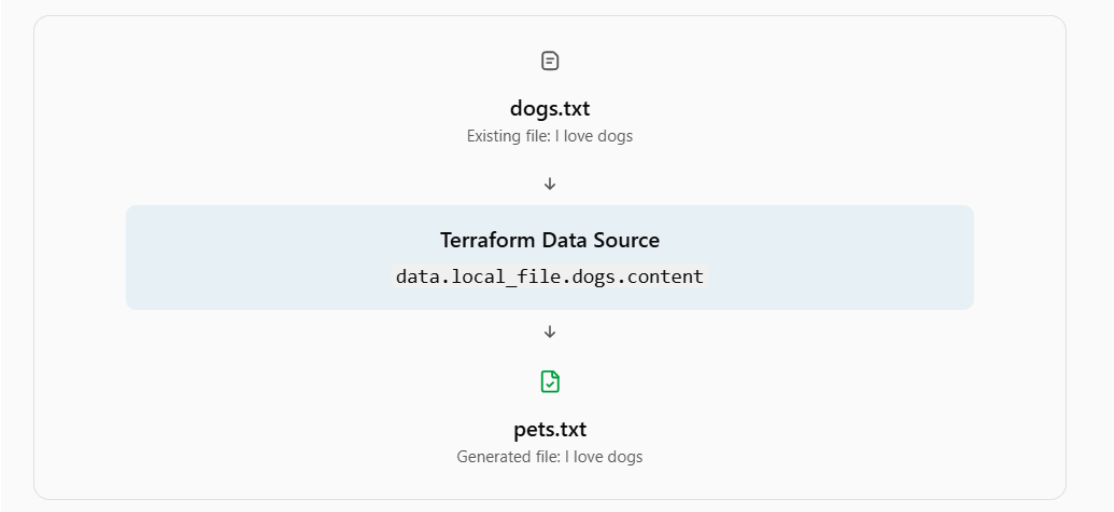
Existing file: I love dogs

Terraform Data Source

`data.local_file.dogs.content`

pets.txt

Generated file: I love dogs



2. Create a new folder in VS Code

You're currently in:

`C:\Users\HP\terraform-04\task-2a-provider`

Execute:

```
cd ..
mkdir task-2b-data-source
cd task-2b-data-source
```

3. Create an existing file manually

Execute in the PowerShell terminal:

```
Set-Content -Path dogs.txt -Value "I love dogs" -Encoding ascii
Get-Content dogs.txt
```

Expected output:

```
I love dogs
```

4. Create `main.tf`

Paste this code:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "2.4.0"
    }
  }
}

# Read an existing file
data "local_file" "dogs" {
  filename = "${path.module}/dogs.txt"
}

# Create a new file using data source content
resource "local_file" "pets" {
  filename = "${path.module}/pets.txt"
  content  = data.local_file.dogs.content
}
```

Understand these two blocks:

Block	Purpose

`data "local_file" "dogs"`	Reads the existing `dogs.txt`
`resource "local_file" "pets"`	Creates `pets.txt` using the content read by the data source

5. Execute Terraform

Run one by one:

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

Enter `yes` after reviewing the plan.

6. Verify the results

Get-Content dogs.txt
Get-Content pets.txt
terraform state list

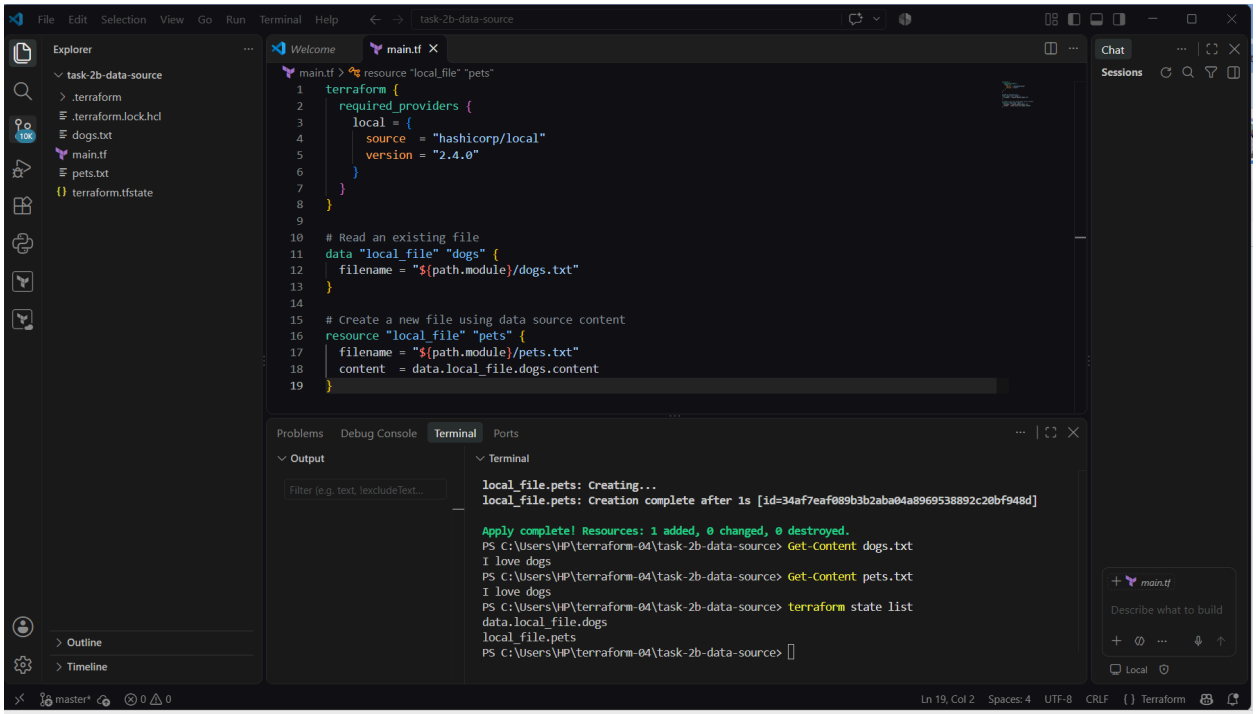
Expected output:

PS> Get-Content dogs.txt
I love dogs

PS> Get-Content pets.txt
I love dogs

Expected Terraform state entries:

data.local_file.dogs
local_file.pets



Task 2C — Terraform `count` Meta-Argument

1. What is `count`?

The `count` meta-argument allows Terraform to create multiple instances of a resource using a single resource block.

Example: Instead of writing three separate `local_file` resource blocks, we can use `count = 3` to create three files.



2. Create a new folder

Your current location is:

`C:\Users\HP\terraform-04\task-2b-data-source`

Execute:

```
cd ..
mkdir task-2c-count
cd task-2c-count
code .
```

3. Create `variables.tf`

```
variable "file_names" {
  type = list(string)

  default = [
    "pets.txt",
    "dogs.txt",
    "cats.txt"
  ]
}
```

Explanation: We are creating a list containing three file names.

Remember that list indexing starts from `0`, not `1`.

4. Create `main.tf`

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "2.4.0"
    }
  }
}

resource "local_file" "my_files" {
  count = 3

  filename = "path.module/{var.file_names[count.index]}"
```

```
  content  = "Created by Terraform using count index ${count.index}"
}
```

Important code explanation

Code	Purpose
`count = 3`	Creates 3 resource instances
`count.index`	Gives index numbers 0, 1, 2
`var.file_names[count.index]`	Selects the filename at that index
`path.module`	Current Terraform module directory
`content`	Text written inside each file

Why use `count.index` in the filename?

If all three instances write to exactly the same filename, they can overwrite one another. Giving each instance a different filename avoids that problem.

5. Execute Terraform

In VS Code PowerShell, run:

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

Enter `yes` after reviewing the plan.

Expected result

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

6. Verify the files

Execute:

terraform state list
Get-ChildItem *.txt
Get-Content pets.txt
Get-Content dogs.txt
Get-Content cats.txt

Expected state resources:

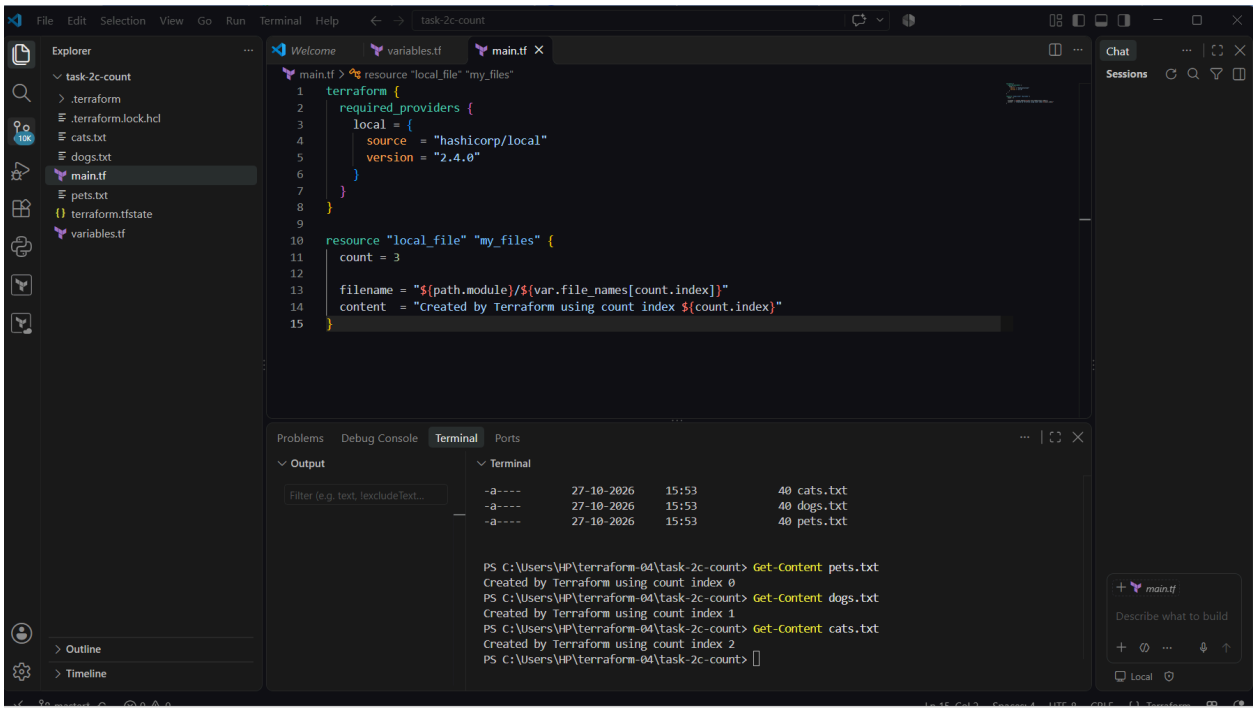
local_file.my_files[0]
local_file.my_files[1]
local_file.my_files[2]

Expected example:

PS> Get-Content pets.txt
Created by Terraform using count index 0

PS> Get-Content dogs.txt
Created by Terraform using count index 1

PS> Get-Content cats.txt
Created by Terraform using count index 2



Task 2D: `length()` and `for_each`

Part A — Terraform `length()` Function

Currently, your `main.tf` contains:

count = 3

The problem is that `3` is hardcoded. If we add or remove filenames, we would need to update the count manually.

Step 1 — Modify your existing `main.tf`

Stay in:

`C:\Users\HP\terraform-04\task-2c-count`

Change only this line:

count = length(var.file_names)

Explanation:

- `length()` counts the elements in the list.
- `var.file_names` contains three filenames.
- Terraform automatically calculates `count = 3`.

Step 2 — Verify

terraform fmt
terraform validate
terraform plan

Expected result:

No changes. Your infrastructure matches the configuration.

Because the list still contains three names, Terraform should not need to modify anything.

Now your code is dynamic. If you add a fourth filename, Terraform can automatically create a fourth resource instance.

Part B — Terraform `for_each` Meta-Argument

1. What is `for_each`?

Like `count`, `for_each` creates multiple resource instances using one resource block.

Main difference:

- `count` identifies resources using numbers: `[0]`, `[1]`, `[2]`.
- `for_each` identifies resources using keys, such as `["pets.txt"]`.

count

Index-based

`my_files[0]`

`my_files[1]`

`my_files[2]`

for_each

Key-based

`by_name["pets.txt"]`

`by_name["dogs.txt"]`

`by_name["cats.txt"]`

Step 1 — Create a new folder

From your current VS Code PowerShell terminal:

```
cd ..
mkdir task-2d-foreach
cd task-2d-foreach
code .
```

Step 2 — Create `variables.tf`

```
variable "file_names" {
  type = list(string)

  default = [
    "pets.txt",
    "dogs.txt",
    "cats.txt"
  ]
}
```

Step 3 — Create `main.tf`

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "2.4.0"
    }
  }
}

resource "local_file" "by_name" {
  for_each = toset(var.file_names)
```

```
  filename = "path.module/{each.value}"
  content  = "Created using for_each: ${each.value}"
}
```

Step 4 — Understand the important code

Terraform code	Purpose
`for_each`	Creates resources using distinct keys
`toset()`	Converts the list into a set of unique values
`each.value`	Retrieves the current filename
`path.module`	Gets the current module directory

Important: `for_each` directly supports maps and sets of strings, not lists. That's why we use `toset(var.file_names)`.

Step 5 — Execute Terraform

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

Enter `yes` after reviewing the plan.

Expected result:

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Step 6 — Verify

Execute:

terraform state list
Get-ChildItem *.txt
Get-Content pets.txt
Get-Content dogs.txt
Get-Content cats.txt

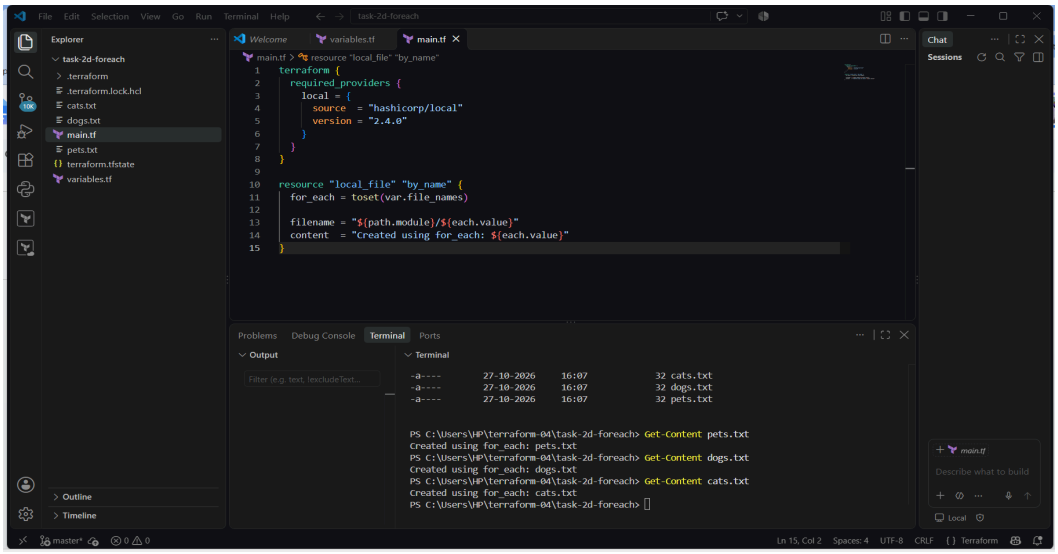
Expected state:

local_file.by_name["cats.txt"]
local_file.by_name["dogs.txt"]
local_file.by_name["pets.txt"]

Expected file content example:

PS> Get-Content pets.txt

Created using for_each: pets.txt

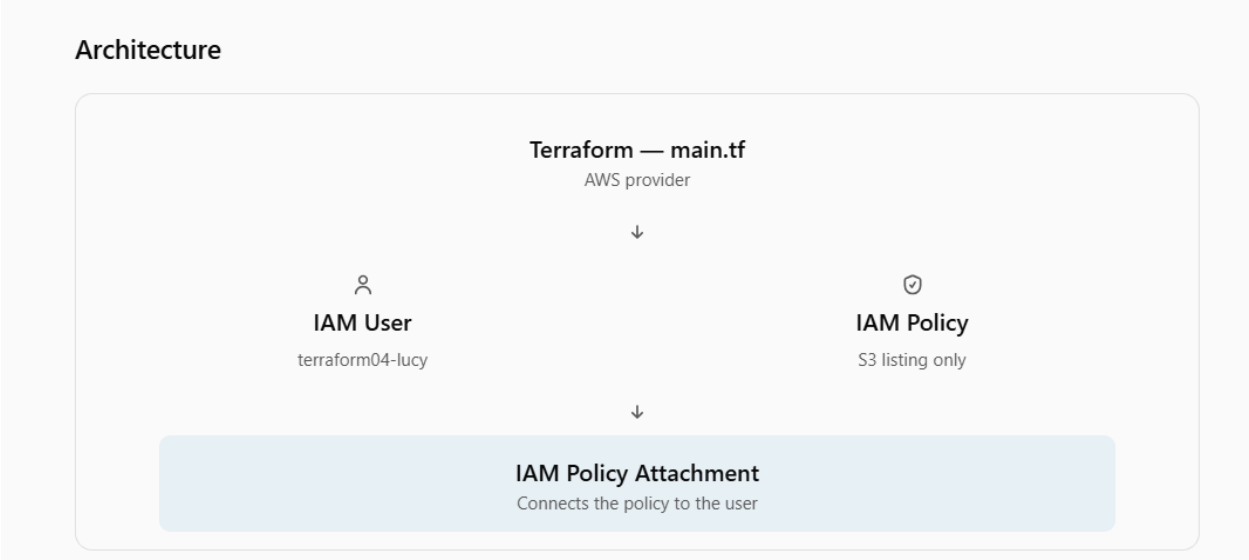


Q: When should we use `count` and when should we use `for_each`?

Answer: We use `count` when resources can be identified by numeric indexes. We use `for_each` when resources have meaningful unique keys, such as filenames, subnet names or environment names. `for_each` generally makes updates easier when collection items are added or removed because resource identity is tied to the key rather than its position.

Task 2E — Create AWS IAM Resources Using Terraform

the AWS practical creates an IAM user, an IAM policy, and an attachment connecting them. We'll follow that structure, but use a limited read-only S3 listing policy instead of administrator permissions. This is safer for a learning account.



Task: Create AWS IAM User, IAM Policy and Policy Attachment Using Terraform

Objective

To use Terraform Infrastructure as Code (IaC) to create and manage AWS IAM resources.

In this practical, we performed the following operations:

1. Configured AWS CLI on Windows.
2. Authenticated AWS CLI using an IAM profile.
3. Created an IAM user using Terraform.
4. Created a custom IAM policy.
5. Attached the IAM policy to the IAM user.
6. Verified the resources using Terraform, AWS CLI and AWS Console.
7. Destroyed all three resources using Terraform.

---

2. Environment Details

Component	Details
Operating System	Windows
Editor	Visual Studio Code
Terminal	PowerShell
IaC Tool	Terraform
Provider	hashicorp/aws
Installed AWS Provider Version	6.70.0
AWS CLI	AWS CLI v2
AWS Region	us-east-1
AWS CLI Profile	terraform-lab
IAM User	terraform04-lucy
IAM Policy	terraform04-s3-listonly

Project Directory:

C:\Users\HP\terraform-04\task-2e-aws-iam

---

3. Architecture

Windows Laptop

|

| VS Code + PowerShell

|

v

Terraform Configuration (main.tf)

|

v

AWS Provider

|

| AWS CLI Authentication

| Profile: terraform-lab

|

v

AWS IAM Service

|

+-- IAM User

| terraform04-lucy

|

+-- IAM Policy

| terraform04-s3-listonly

|

+-- IAM Policy Attachment

  Attaches Policy to User

Step 1 — Create the Terraform Project Directory

Opened VS Code and created a separate directory for the AWS IAM practical.

Executed in PowerShell:

cd C:\Users\HP\terraform-04

mkdir task-2e-aws-iam

cd task-2e-aws-iam

code .

Purpose: Keep the AWS IAM configuration and Terraform state separate from the previous Terraform-04 practicals.

Step 2 — Install and Verify AWS CLI

Initially, when executing:

aws --version

We received:

aws : The term 'aws' is not recognized

Root Cause: AWS CLI was not installed or was not accessible through the Windows PATH.

Resolution:

1. Downloaded the official AWS CLI v2 Windows installer.
2. Installed AWS CLI.
3. Reopened VS Code and PowerShell.
4. Verified the installation.

Command:

aws --version

Result: AWS CLI became available for executing AWS commands.

Step 3 — Configure AWS CLI Authentication

Configured a named AWS CLI profile to authenticate Terraform requests.

Executed:

aws configure --profile terraform-lab

Entered the following values privately:

AWS Access Key ID: [Authorized access key]

AWS Secret Access Key: [Matching secret key]

Default region name: us-east-1

Default output format: json

Security Note: AWS credentials were not hardcoded in Terraform configuration files.

Initial Error

During authentication testing, we encountered:

Partial credentials found in shared-credentials-file,

missing: aws_secret_access_key

Root Cause: The AWS CLI profile contained an Access Key ID but did not contain its matching Secret Access Key.

Resolution: Reconfigured the profile with a complete, valid credential pair.

Authentication Verification

Executed:

aws sts get-caller-identity --profile terraform-lab

Then selected the profile for Terraform in the PowerShell session:

$env:AWS_PROFILE = "terraform-lab"

Purpose: Allow Terraform to use the configured AWS credentials without storing them in main.tf.

Step 4 — Create the Terraform Configuration File

Created a file named:

main.tf

Added the following Terraform configuration.

Complete Terraform Code

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Resource 1: Create IAM User

resource "aws_iam_user" "lab_user" {
  name = "terraform04-lucy"
```

```
  tags = {
    Purpose = "Terraform04-Lab"
  }
}

# Resource 2: Create IAM Policy

resource "aws_iam_policy" "lab_policy" {
  name        = "terraform04-s3-listonly"
  description = "Read-only S3 bucket listing for Terraform lab"

  policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect   = "Allow"
        Action   = ["s3:ListAllMyBuckets"]
```

```
      Resource = "*"
    }
  ]
 })
}

# Resource 3: Attach IAM Policy to User

resource "aws_iam_user_policy_attachment" "lab_attach" {
 user       = aws_iam_user.lab_user.name
 policy_arn = aws_iam_policy.lab_policy.arn
}
```

Code Explanation

Terraform Block

```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }
```

```
  }

}
```

Specifies the required AWS provider and compatible version range.

~> 6.0 permits compatible provider releases within the 6.x series.

Provider Block

```
provider "aws" {

  region = "us-east-1"

}
```

Configures the AWS provider for the us-east-1 region.

Although IAM is a global AWS service, the provider uses the configured region for AWS API connectivity.

IAM User Resource

```
resource "aws_iam_user" "lab_user"
```

Creates the AWS IAM user named terraform04-lucy.

IAM Policy Resource

```
resource "aws_iam_policy" "lab_policy"
```

Creates a custom policy that allows listing S3 buckets.

The policy contains only the s3:ListAllMyBuckets permission; it does not provide administrator access.

IAM Policy Attachment

```
resource "aws_iam_user_policy_attachment" "lab_attach"
```

Attaches the newly created policy to the IAM user.

The expressions below create implicit Terraform dependencies:

aws_iam_user.lab_user.name

aws_iam_policy.lab_policy.arn

Terraform understands that the user and policy must exist before the attachment is created.

jsonencode() Function

The jsonencode() function converts Terraform expressions into JSON.

It is useful for writing AWS IAM policies without manually escaping JSON strings.

Step 5 — Initialize Terraform

Executed:

terraform init

Actual Result

Initializing the backend...

Initializing provider plugins...

- Finding hashicorp/aws versions matching "~> 6.0"...

- Installing hashicorp/aws v6.70.0...

- Installed hashicorp/aws v6.70.0

Terraform has been successfully initialized!

Explanation:

- Initialized the working directory.
- Downloaded the required AWS provider.
- Created .terraform.lock.hcl.
- Prepared Terraform for execution.

Status: SUCCESS

Step 6 — Format and Validate Terraform Code

Executed:

terraform fmt

terraform validate

Actual Validation Output

Success! The configuration is valid.

Explanation:

terraform fmt formats the Terraform configuration.

terraform validate checks the syntax and internal consistency of the configuration.

Status: SUCCESS

Step 7 — Execute Terraform Plan

Executed:

terraform plan

Initial Troubleshooting

During the first attempt, Terraform was interrupted while communicating with AWS STS.

Error:

Stopping operation...

Interrupt received.

Error: Retrieving AWS account details:

validating provider credentials:

retrieving caller identity from STS:

request canceled, context canceled

Explanation: Terraform received an interrupt before the AWS provider's identity check completed. This did not establish that the credentials were invalid.

Resolution:

1. Verified AWS CLI authentication.
2. Confirmed the correct AWS profile.
3. Re-ran terraform plan.
4. Allowed the command to finish normally.

Successful Terraform Plan

Terraform displayed three resources to be created:

aws_iam_policy.lab_policy

aws_iam_user.lab_user

aws_iam_user_policy_attachment.lab_attach

Actual Plan Result

Plan: 3 to add, 0 to change, 0 to destroy.

Status: SUCCESS

Step 8 — Create AWS IAM Resources

Executed:

terraform apply

Reviewed the Terraform execution plan.

Entered:

yes

Actual Result

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Explanation:

Terraform successfully created:

1. IAM user — terraform04-lucy
2. IAM policy — terraform04-s3-listonly
3. IAM policy attachment

Status: SUCCESS

Step 9 — Verify Terraform State

Executed:

terraform state list

Actual Output

aws_iam_policy.lab_policy

aws_iam_user.lab_user

aws_iam_user_policy_attachment.lab_attach

Explanation:

Terraform state recorded all three AWS IAM resources.

This confirms that Terraform was tracking the resources it created.

Status: SUCCESS

Step 10 — Verify IAM User Using AWS CLI

Executed:

aws iam get-user --user-name terraform04-lucy --profile terraform-lab

Verified Details

UserName: terraform04-lucy

Path: /

Tag:

Purpose = Terraform04-Lab

Result: AWS CLI confirmed the IAM user existed in the AWS account.

Step 11 — Verify IAM Policy Attachment

Executed:

aws iam list-attached-user-policies --user-name terraform04-lucy --profile terraform-lab

Actual Result


```
{
 "AttachedPolicies": [
   {
    "PolicyName": "terraform04-s3-listonly",
    "PolicyArn": "arn:aws:iam::<account-id>:policy/terraform04-s3-listonly"
   }
 ]
}
```


The account ID above is intentionally masked for documentation.

Result: The policy was successfully attached to the IAM user.

Step 12 — Verify Using AWS Management Console

Opened AWS Console and navigated to:

AWS Console → IAM → Users

Verified that the following IAM user existed:

terraform04-lucy

The screenshot also showed the existing administrative IAM user.

Then checked the policy attachment using the AWS CLI.

Result: Terraform-created IAM resources were successfully verified in AWS.

Step 13 — Destroy the AWS Resources

After completing the practical and collecting evidence, we cleaned up the AWS resources.

Preview Cleanup

Command:

terraform plan -destroy

The intended cleanup targeted the three lab resources.

Execute Cleanup

terraform destroy

Entered:

yes

Actual Result

Destroy complete! Resources: 3 destroyed.

Verify State After Destroy

Executed:

terraform state list

Actual Output

No resources listed.

Explanation:

All three managed AWS IAM resources were successfully destroyed.

Terraform state no longer contained those resource instances.

Status: CLEANUP SUCCESSFUL

Step 14 — Important Commands Used

Command	Purpose
aws --version	Verify AWS CLI installation
aws configure --profile terraform-lab	Configure AWS credentials
aws sts get-caller-identity	Verify AWS identity
terraform init	Initialize provider
terraform fmt	Format HCL files
terraform validate	Validate configuration
terraform plan	Preview resource changes
terraform apply	Create IAM resources
terraform state list	List tracked resources
aws iam get-user	Verify the IAM user
aws iam list-attached-user-policies	Verify attached policy

terraform plan -destroy	Preview deletion
terraform destroy	Delete managed infrastructure

Step 15 — Real-Time DevOps Use Case

Scenario: A development team requires a new IAM user with limited AWS access.

Manual Approach:

An administrator logs in to AWS Console, creates the user, creates a custom policy and attaches it manually.

Terraform Approach:

A DevOps engineer creates the IAM user, custom policy and attachment using Terraform configuration files.

Benefits

- Infrastructure as Code.
- Consistent IAM configuration.
- Repeatable infrastructure provisioning.
- Easier review through Terraform plans.
- Resource tracking using Terraform state.
- Automated cleanup when resources are no longer required.

In production, temporary IAM role credentials are generally preferable to creating long-term IAM user credentials for applications and automation.

Q1. What is Terraform?

Terraform is an Infrastructure as Code tool used to define, provision and manage infrastructure through configuration files.

Q2. What is the purpose of the AWS provider?

The AWS provider allows Terraform to communicate with AWS APIs and manage AWS resources.

Q3. What is the difference between terraform plan and terraform apply?

terraform plan previews proposed changes. terraform apply executes the approved changes.

Q4. Why do we use aws_iam_user?

To create and manage AWS IAM users using Terraform.

Q5. What is aws_iam_policy?

It is a Terraform resource used to create a customer-managed IAM policy.

Q6. Why do we use aws_iam_user_policy_attachment?

It attaches a managed IAM policy to an IAM user.

Q7. What is jsonencode()?

It converts a Terraform value into a JSON string, which is useful for AWS IAM policy documents.

Q8. Why should we avoid hardcoding AWS credentials?

Hardcoded credentials can be exposed in source code and Git repositories. Safer options include AWS CLI profiles, SSO and IAM role credentials.

Q9. What is Terraform state?

Terraform state records information about managed resources and maps Terraform resource addresses to real infrastructure.

Q10. What happens when we execute terraform destroy?

Terraform prepares and applies deletion operations for the managed resources in the current configuration and state.

Conclusion

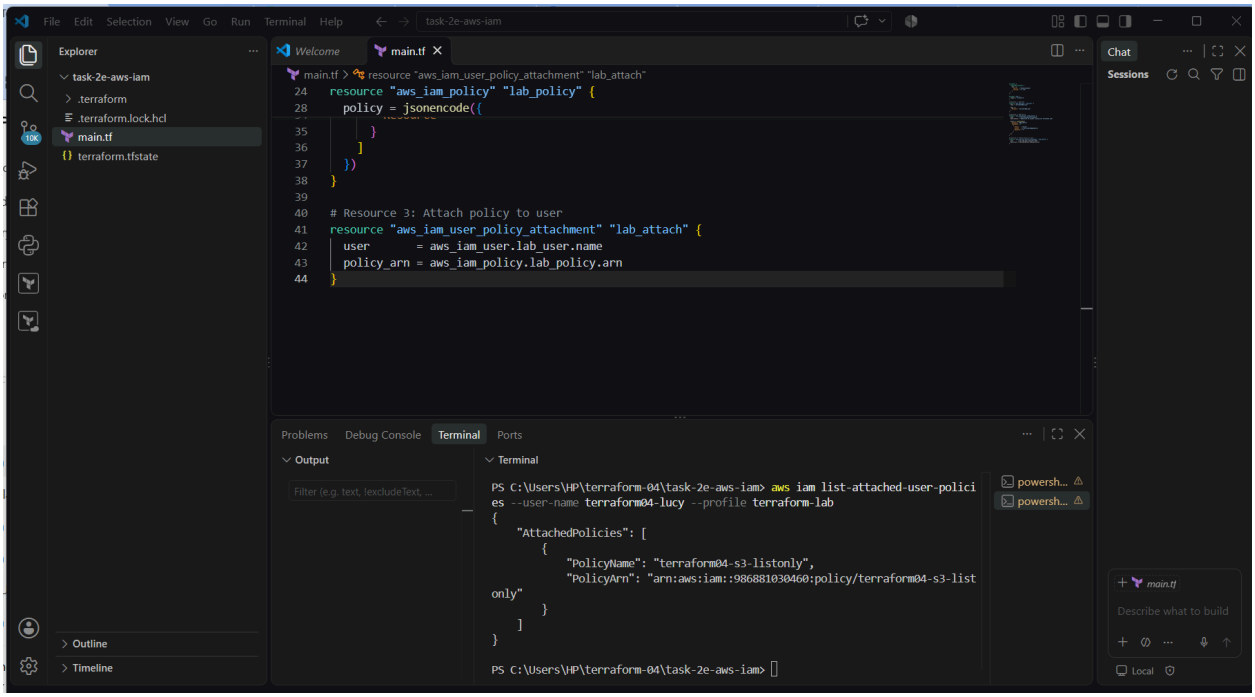
Successfully completed Terraform-04 Task 2E by creating and managing AWS IAM resources using Terraform from Windows VS Code.

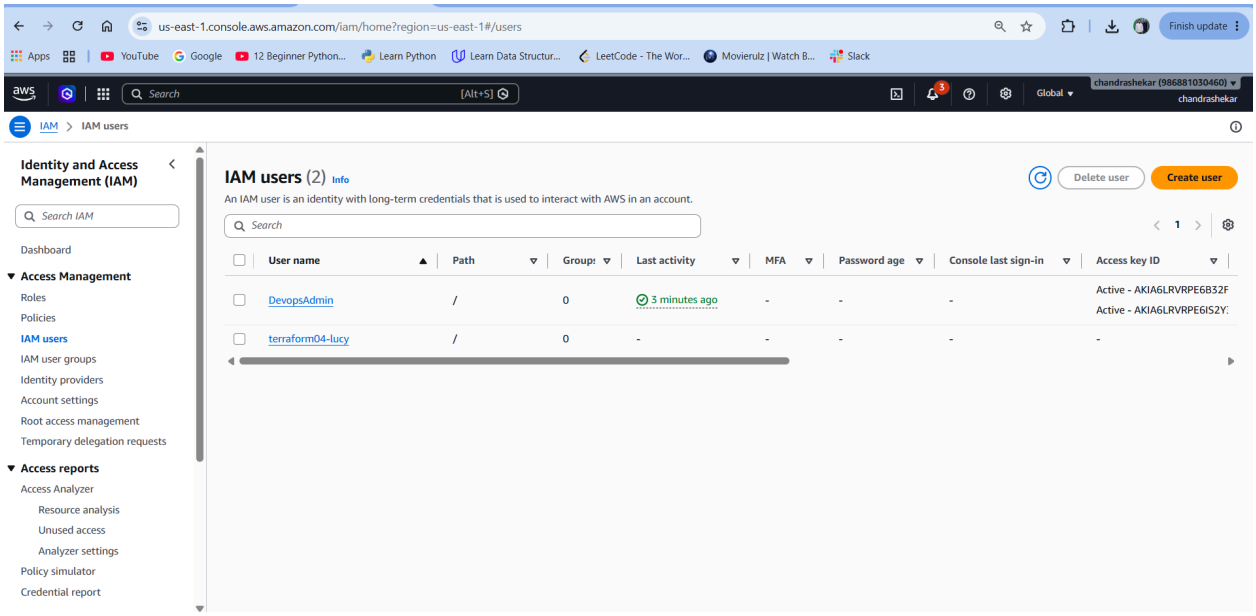
Configured AWS CLI authentication, initialized the AWS provider, validated the Terraform configuration, and executed Terraform to create an IAM user, a custom policy and a policy attachment.

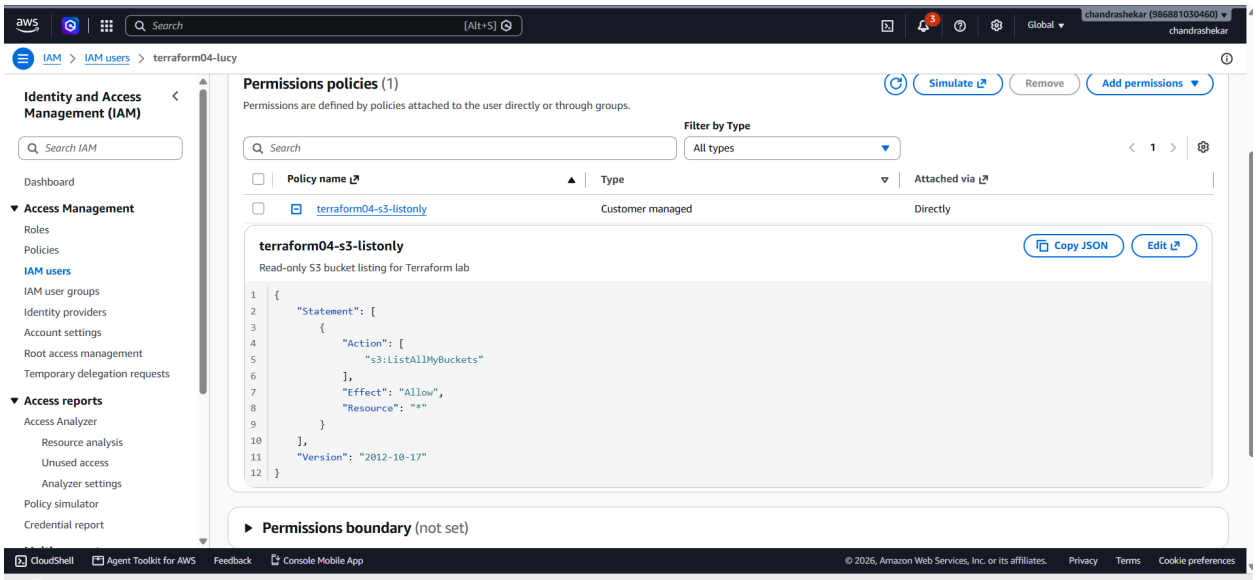
Verified the resources using Terraform state, AWS CLI and AWS Management Console.

Finally, executed terraform destroy and confirmed that all three IAM resources were removed.

Final Status: TERRAFORM-04 TASK 2E COMPLETED SUCCESSFULLY.







3). Integrate Terraform in Jenkins using the Terraform plugin.

Jenkins Terraform Plugin Integration

1. Objective

Install and configure Terraform on a Jenkins EC2 server, integrate Terraform with Jenkins using the Terraform plugin, create Jenkins jobs, and verify successful Terraform execution.

2. Environment Details

Component	Details
Cloud Provider	AWS
Server	EC2 Ubuntu 24.04 LTS
Private IP	172.31.93.182
Public IP at time of task	3.86.29.154
Jenkins Version	2.580.1
Jenkins Port	8080
Terraform Version	1.17.0
Terraform Path	`/usr/bin/terraform`
Git Version	2.43.0
Terraform Jenkins Plugin	Terraform (terraform)

3. Step 1 — Prepare Jenkins Server

Launched a new Ubuntu EC2 instance and installed Java 21 and Jenkins LTS.

Verified the Jenkins service:

sudo systemctl status jenkins

Checked the Jenkins port:

sudo ss -tulnp | grep 8080

Verified the Jenkins application locally:

curl -I http://localhost:8080

Configured the EC2 Security Group to allow TCP port 8080 from the administrator's public IP.

Accessed Jenkins using:

http://3.86.29.154:8080

Result: Jenkins dashboard opened successfully.

4. Step 2 — Install Terraform

Installed required packages:

```
sudo apt update
sudo apt install -y wget gnupg lsb-release
```

Downloaded and configured the official HashiCorp repository signing key.

Added the HashiCorp APT repository for Ubuntu 24.04.

Installed Terraform:

```
sudo apt update
sudo apt install -y terraform
```

Verified installation:

```
terraform version
```

Output:

```
Terraform v1.17.0
on linux_amd64
```

Verified installation path:

```
which terraform
```

Output:

```
/usr/bin/terraform
```

5. Step 3 — Verify Jenkins User Permissions

Verified that the Jenkins service account can execute Terraform:

```
sudo -u jenkins terraform version
```

Output:

```
Terraform v1.17.0
on linux_amd64
```

Verified Git:

git --version
sudo -u jenkins git --version

Output:

git version 2.43.0

Result: Terraform and Git were accessible to the Jenkins user.

6. Step 4 — Install Terraform Jenkins Plugin

Opened:

Jenkins Dashboard → Manage Jenkins → Plugins → Available Plugins

Searched for:

Terraform

Selected and installed the Terraform plugin.

Verified that it appeared under Installed Plugins.

7. Step 5 — Configure Terraform Tool in Jenkins

Navigated to:

Manage Jenkins → Tools → Terraform installations

Configured:

Setting	Value
Name	Terraform-CLI
Install automatically	Unchecked
Installation directory	/usr/bin

Important: Disabled automatic installation because Terraform was already installed on the Ubuntu server.

The automatic installer displayed a macOS (Darwin) package, which was inappropriate for our Linux server.

Clicked **Save**.

8. Step 6 — Create Terraform Pipeline Test Job

Navigated to:

Jenkins Dashboard → New Item

Entered job name:

terraform-04-test

Selected:

Pipeline

Added the following Jenkinsfile:

```
pipeline {
    agent any

    stages {
        stage('Verify Terraform') {
            steps {
                sh '/usr/bin/terraform version'
            }
        }
    }
}
```

Clicked **Save → Build Now**.

Console Output:

```
[Pipeline] Start of Pipeline
[Pipeline] stage
[Pipeline] { (Verify Terraform)
[Pipeline] sh

+ /usr/bin/terraform version

Terraform v1.17.0
on linux_amd64

[Pipeline] End of Pipeline

Finished: SUCCESS
```

Result: Jenkins successfully executed Terraform from the Pipeline.

This verified CLI integration but did not yet exercise the Terraform plugin.

9. Step 7 — Create Jenkins Terraform Plugin Job

Navigated to:

Jenkins Dashboard → New Item

Entered job name:

terraform-04-plugin

Selected:

Freestyle project

Opened job configuration.

Under **Build Environment**, enabled the Terraform plugin configuration.

Configured the Terraform installation as:

Terraform-CLI

Used a minimal Terraform configuration for testing:

```
terraform {
  required_version = ">= 1.0.0"
}
```

Saved the configuration.

10. Step 8 — Execute Terraform Plugin Job

Clicked:

Build Now → Build History → Console Output

Jenkins executed:

/usr/bin/terraform get

Then:

/usr/bin/terraform apply -input=false
-state=/var/lib/jenkins/workspace/terraform-04-plugin/terraform-plugin/terraform-plugin.tfstate

Actual Console Output:

Started by user admin
Running as SYSTEM

Building in workspace
/var/lib/jenkins/workspace/terraform-04-plugin

[terraform-plugin] $ /usr/bin/terraform get

[terraform-plugin] $ /usr/bin/terraform apply
-input=false -state=...

Warning: Deprecated flag: -state

No changes.
Your infrastructure matches the configuration.

Apply complete!
Resources: 0 added, 0 changed, 0 destroyed.

Finished: SUCCESS

Result: Jenkins successfully executed Terraform using the Terraform plugin.

11. Step 9 — Troubleshooting Performed

Issue 1: Jenkins not accessible on old EC2 instance

Jenkins was running locally, but browser access to the old instance failed.

Resolution: Continued the assignment on a new accessible EC2 instance.

Issue 2: Terraform command not found

terraform: command not found

Resolution: Installed Terraform using the official HashiCorp APT repository.

Issue 3: HashiCorp GPG key error

NO_PUBKEY FC9CA96ACA026560

Resolution: Downloaded and installed the correct current HashiCorp repository signing key.

Issue 4: Incorrect Terraform automatic installer

Jenkins showed:

Terraform 1.17.0 darwin (amd64)

Resolution: Disabled automatic installation and configured the existing Linux installation at `/usr/bin`.

Issue 5: Deprecated Terraform flag

Warning: Deprecated flag: -state

Explanation: The older Jenkins Terraform plugin passes a deprecated `-state` argument. Terraform 1.17.0 still accepted it for this task.

Result: Build succeeded despite the warning.

12. Final Verification

Verification	Result
Jenkins installation	SUCCESS
Terraform installation	SUCCESS
Git installation	SUCCESS
Jenkins Terraform CLI access	SUCCESS
Terraform plugin installation	SUCCESS
Terraform tool configuration	SUCCESS
Pipeline test build	SUCCESS
Freestyle plugin build	SUCCESS
Terraform apply	SUCCESS
AWS resources created	0

13. Key Learning Points

- Jenkins automates infrastructure operations by executing Terraform commands.
- Jenkins plugins can integrate Terraform with Freestyle jobs.

- The Jenkins service account needs access to the Terraform executable.
- Terraform installation paths must match the operating system.
- `terraform get` downloads and updates Terraform modules.
- `terraform apply` applies infrastructure changes defined in Terraform configuration.
- Deprecated plugin arguments can generate warnings even when a job succeeds.
- Modern production pipelines should use current Terraform CLI commands with `init`, `validate`, `plan`, review and controlled `apply`.
- Terraform state and credentials must be managed securely for real infrastructure deployments.

14. Screenshots for Assignment Submission

Attach the following screenshots from Jenkins and your terminal:

1. Terraform version and installation path.
2. Terraform tool configuration in Jenkins.
3. `terraform-04-test` Pipeline Console Output showing SUCCESS.
4. `terraform-04-plugin` Freestyle job configuration.
5. Terraform plugin Console Output showing `Apply complete`.
6. Jenkins Build History showing SUCCESS.

15. Conclusion

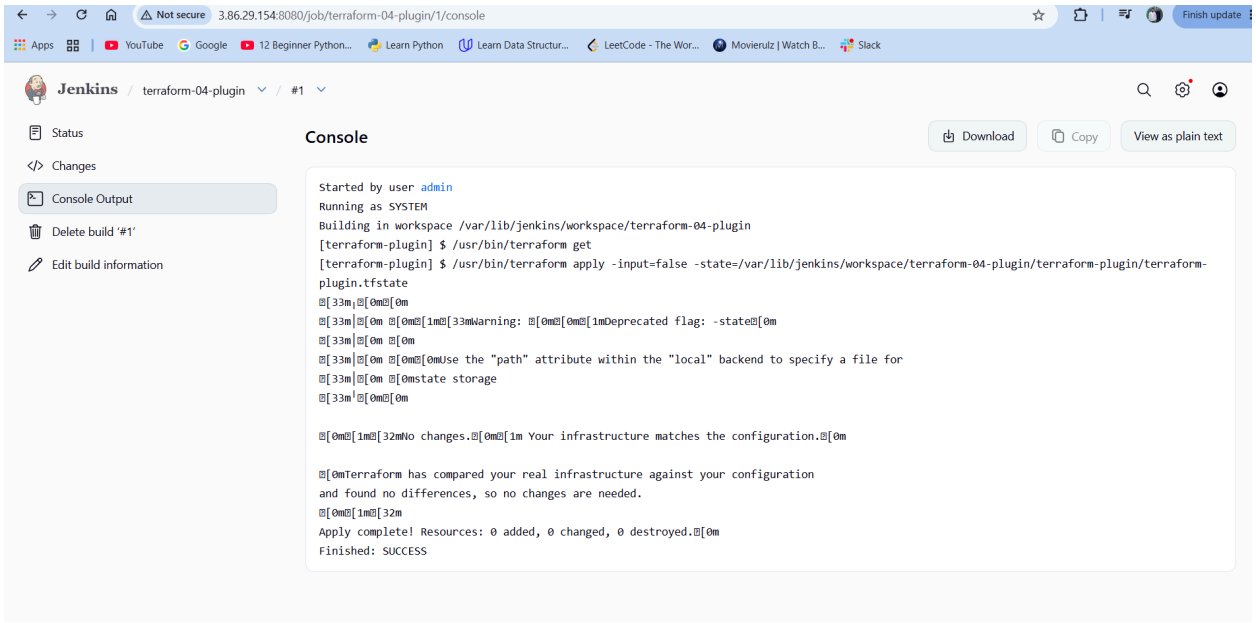
Successfully installed and configured Terraform 1.17.0 on the Ubuntu Jenkins EC2 server.

Verified Terraform execution through both a Jenkins Pipeline and the Terraform Jenkins plugin using a Freestyle job.

Both Jenkins jobs completed successfully.

The plugin test applied a minimal Terraform configuration with zero resources created, changed or destroyed. This demonstrated successful integration without provisioning AWS infrastructure.

FINAL TASK STATUS: TERRAFORM-04 TASK 3 — COMPLETED SUCCESSFULLY.



4). Create a CI/CD pipeline for a Nodejs Application: github.com/betawins/Trading-UI.git

CI/CD Pipeline for a Node.js (React) Application Using Jenkins and Nginx

---

1. Objective

Create a CI/CD pipeline for a Node.js-based React application using Jenkins.

The pipeline should automatically perform the following activities when executed:

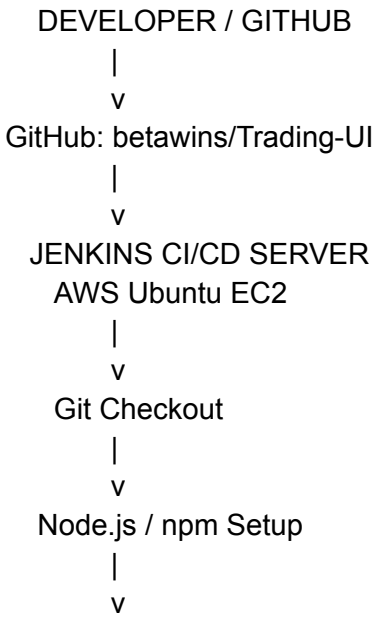
1. Download source code from GitHub.
2. Verify Node.js and npm.
3. Install project dependencies.
4. Build the React application.
5. Verify production build artifacts.
6. Deploy the application to Nginx.
7. Verify application access from a web browser.

2. Environment Details

Component	Configuration
Cloud Provider	AWS

Operating System	Ubuntu 24.04 LTS
EC2 Public IP at deployment	3.86.29.154
EC2 Private IP	172.31.93.182
Jenkins	2.580.1
Jenkins Port	8080
Node.js	v22.22.0
npm	10.9.4
Git	2.43.0
Application Framework	React
Web Server	Nginx
Application Port	8082
Repository	https://github.com/betawins/Trading-UI.git
Git Branch	master
Jenkins Job	terraform-04-trading-ui

3. Architecture Diagram



Install Dependencies
|
v
React Build
|
v
Verify build/index.html
|
v
Deploy using rsync
|
v
NGINX SERVER
Port 8082 (HTTP)
|
v
WEB BROWSER / USER

Explanation: Jenkins downloads the React application from GitHub, installs dependencies, generates optimized static files and copies them to the Nginx deployment directory. Nginx serves the application to browser users.

4. Step 1 — Prepare the Jenkins EC2 Server

Used the existing Ubuntu EC2 instance with Jenkins already installed.

Verified Jenkins:

sudo systemctl status jenkins

Verified Jenkins listening port:

sudo ss -tulnp | grep 8080

Accessed Jenkins using:

http://3.86.29.154:8080

Result: Jenkins dashboard opened successfully.

5. Step 2 — Install and Configure NodeJS Plugin

Opened:

Jenkins Dashboard → Manage Jenkins → Plugins → Available Plugins

Searched for:

NodeJS

Installed the NodeJS plugin.

Navigated to:

Manage Jenkins → Tools → NodeJS installations → Add NodeJS

Configured:

Setting	Value
Name	NodeJS-22
Install automatically	Enabled
Installer	Install from nodejs.org
Version	Node.js 22.22.0

Clicked **Save**.

Important: Selected Linux-compatible Node.js 22 instead of the initially installed Node.js 26.

6. Step 3 — Create Jenkins Pipeline Job

Opened:

Jenkins → New Item

Entered the following settings:

Field	Value
Item name	terraform-04-trading-ui
Job type	Pipeline
Pipeline definition	Pipeline script

The application repository used was:

https://github.com/betawins/Trading-UI.git

Branch:

master

7. Step 4 — Verify Node.js Installation

Created the first pipeline containing Node.js verification and Git checkout stages.

Commands:

```
node --version
npm --version
```

Successful output:

```
v22.22.0
10.9.4
```

Result: Jenkins successfully executed Node.js and npm.

8. Step 5 — GitHub Source Code Checkout

Configured Jenkins to pull source code using:

```
git branch: 'master',
    url: 'https://github.com/betawins/Trading-UI.git'
```

Jenkins checked out the repository into:

```
/var/lib/jenkins/workspace/terraform-04-trading-ui
```

Verified the following important files:

```
Trading-UI/
├── package.json
├── package-lock.json
├── Jenkinsfile
├── README.md
├── public/
└── src/
```

Result: Git checkout completed successfully.

9. Step 6 — Install Node.js Dependencies

The project uses an older React dependency tree.

Initially tried:

npm ci --legacy-peer-deps --no-audit --no-fund

This failed due to a macOS-only `fsevents` dependency.

Also tried:

npm ci --legacy-peer-deps --omit=optional --no-audit --no-fund

The same platform error occurred.

Finally, used the following command:

```
npm install \
  --package-lock=false \
  --legacy-peer-deps \
  --omit=optional \
  --no-audit \
  --no-fund
```

Explanation:

- `npm install` — Installs application dependencies.
- `--package-lock=false` — Avoids using the outdated lockfile for dependency resolution.
- `--legacy-peer-deps` — Handles legacy peer dependency constraints.
- `--omit=optional` — Skips optional dependencies.
- `--no-audit` — Disables automatic audit checks for this installation.
- `--no-fund` — Disables funding messages.

Result: Dependencies installed successfully.

Production consideration: This workaround can result in different dependency versions between builds. For a real project, update and commit a tested lockfile and use reproducible `npm ci` installations.

10. Step 7 — Build the React Application

Used the command:

npm run build

Configured the build environment:

NODE_OPTIONS=--openssl-legacy-provider
CI=false

This helped the older React/Webpack build system work with the newer Node.js version.

Result: React production build generated successfully.

The output directory was:

/var/lib/jenkins/workspace/terraform-04-trading-ui/build

11. Step 8 — Verify Build Artifacts

Used:

```
test -f build/index.html
ls -lh build/
```

Verified that the following files were generated:

```
build/
├── asset-manifest.json
├── favicon.ico
├── index.html
├── logo192.png
├── logo512.png
├── manifest.json
├── robots.txt
├── service-worker.js
└── static/
```

Result: Production build verification succeeded.

12. Step 9 — Install Nginx

Connected to the Ubuntu EC2 server.

Installed Nginx and rsync:

```
sudo apt update
sudo apt install -y nginx rsync
```

Started Nginx:

```
sudo systemctl enable --now nginx
```

Verified:

sudo systemctl status nginx

Result: Nginx was installed and configured to run.

13. Step 10 — Create Application Deployment Directory

Created the application directory:

sudo mkdir -p /var/www/trading-ui

Assigned ownership to Jenkins:

sudo chown jenkins:www-data /var/www/trading-ui

Configured directory permissions:

sudo chmod 755 /var/www/trading-ui

Verified that Jenkins could write to the directory:

sudo -u jenkins test -w /var/www/trading-ui && echo "Deployment directory ready"

Purpose: Jenkins requires write access to copy production build files into the Nginx document root.

14. Step 11 — Configure Nginx

Created the configuration file:

sudo nano /etc/nginx/sites-available/trading-ui

Added:

```
server {
    listen 8082;
    server_name _;

    root /var/www/trading-ui;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
```

}

Enabled the site:

```
sudo ln -sfn /etc/nginx/sites-available/trading-ui \
  /etc/nginx/sites-enabled/trading-ui
```

Validated Nginx:

```
sudo nginx -t
```

Reloaded Nginx:

```
sudo systemctl reload nginx
```

Explanation: The `try_files` configuration allows client-side React routes to fall back to `index.html`.

15. Step 12 — Configure AWS Security Group

Opened:

AWS Console → EC2 → Instance → Security → Security Group → Inbound Rules

Configured the required access:

Protocol	Port	Purpose
TCP	22	SSH administration
TCP	8080	Jenkins
TCP	8082	Trading-UI application

Restricted administrative access to the administrator's public IP wherever practical.

Result: Application port 8082 was accessible from the browser.

16. Step 13 — Add Deployment Stage in Jenkins

Added the following stage after the successful React build verification:

```
stage('Deploy Application') {
    steps {
```

```
        sh '''
            echo "Deploying Trading-UI to Nginx..."

            rsync -a --delete build/ /var/www/trading-ui/

            test -f /var/www/trading-ui/index.html

            echo "Deployment completed successfully!"
        '''
    }
}
```

Explanation:

- `rsync -a` copies the build files while preserving relevant attributes.
- `--delete` removes files no longer present in the new build.
- `test -f` verifies that the deployed entry point exists.

The deployment stage succeeded without requiring passwordless sudo from Jenkins.

17. Step 14 — Final Jenkinsfile

The following Jenkinsfile represents the successful pipeline configuration used for the task.

```
pipeline {
    agent any

    tools {
        nodejs 'NodeJS-22'
    }

    stages {

        stage('Verify NodeJS') {
            steps {
                sh 'node --version'
                sh 'npm --version'
            }
        }

        stage('Git Checkout') {
            steps {
                git branch: 'master',
                    url: 'https://github.com/betawins/Trading-UI.git'
```

```
    }
}

stage('Install Dependencies') {
    steps {
        sh '''
            npm install \
              --package-lock=false \
              --legacy-peer-deps \
              --omit=optional \
              --no-audit \
              --no-fund
        '''
    }
}

stage('Build Application') {
    steps {
        withEnv([
            'NODE_OPTIONS=--openssl-legacy-provider',
            'CI=false'
        ]) {
            sh 'npm run build'
        }
    }
}

stage('Verify Build') {
    steps {
        sh 'test -f build/index.html'
        sh 'ls -lh build/'
    }
}

stage('Deploy Application') {
    steps {
        sh '''
            echo "Deploying Trading-UI to Nginx..."

            rsync -a --delete build/ /var/www/trading-ui/

            test -f /var/www/trading-ui/index.html

            echo "Deployment completed successfully!"
```

```
            '''
        }
      }
    }

    post {
      success {
        archiveArtifacts artifacts: 'build/**/*',
                         fingerprint: true
        echo 'Trading-UI build completed successfully!'
      }

      failure {
        echo 'Pipeline failed. Check the Console Output.'
      }
    }
}
```

18. Step 15 — Execute Jenkins Pipeline

Opened:

Jenkins → terraform-04-trading-ui → Build Now

Verified all Jenkins pipeline stages.

Pipeline Stage	Result
Tool Install	SUCCESS
Verify NodeJS	SUCCESS
Git Checkout	SUCCESS
Install Dependencies	SUCCESS
Build Application	SUCCESS
Verify Build	SUCCESS
Deploy Application	SUCCESS
Post Actions	SUCCESS

Actual final deployment output:

[Pipeline] { (Deploy Application)

+ echo Deploying Trading-UI to Nginx...
Deploying Trading-UI to Nginx...

+ rsync -a --delete build/ /var/www/trading-ui/

+ test -f /var/www/trading-ui/index.html

+ echo Deployment completed successfully!
Deployment completed successfully!

[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints

Trading-UI build completed successfully!

[Pipeline] End of Pipeline
Finished: SUCCESS

19. Step 16 — Browser Verification

Opened:

http://3.86.29.154:8082/

Observed results:

- Trading-UI application loaded successfully.
- InstaTradE application header was displayed.
- Login page rendered.
- Email input field was displayed.
- Password input field was displayed.
- Login button was displayed.

Result: Deployment and browser access verified successfully.

Limitation: One header image appeared broken. Backend connectivity and actual login authentication were not tested, so they are outside the verified result of this assignment.

20. Troubleshooting — Errors and Solutions

Issue 1: Missing Node.js Library

Error:

node: error while loading shared libraries:
libatomic.so.1: cannot open shared object file

Cause: Missing Linux runtime library.

Solution:

sudo apt install -y libatomic1

Issue 2: Incorrect Node.js Version

Initially Jenkins downloaded Node.js 26 even though the installation was named `NodeJS-22`.

Solution: Updated Manage Jenkins → Tools → NodeJS installations to explicitly select Node.js 22.x.

Verified:

v22.22.0

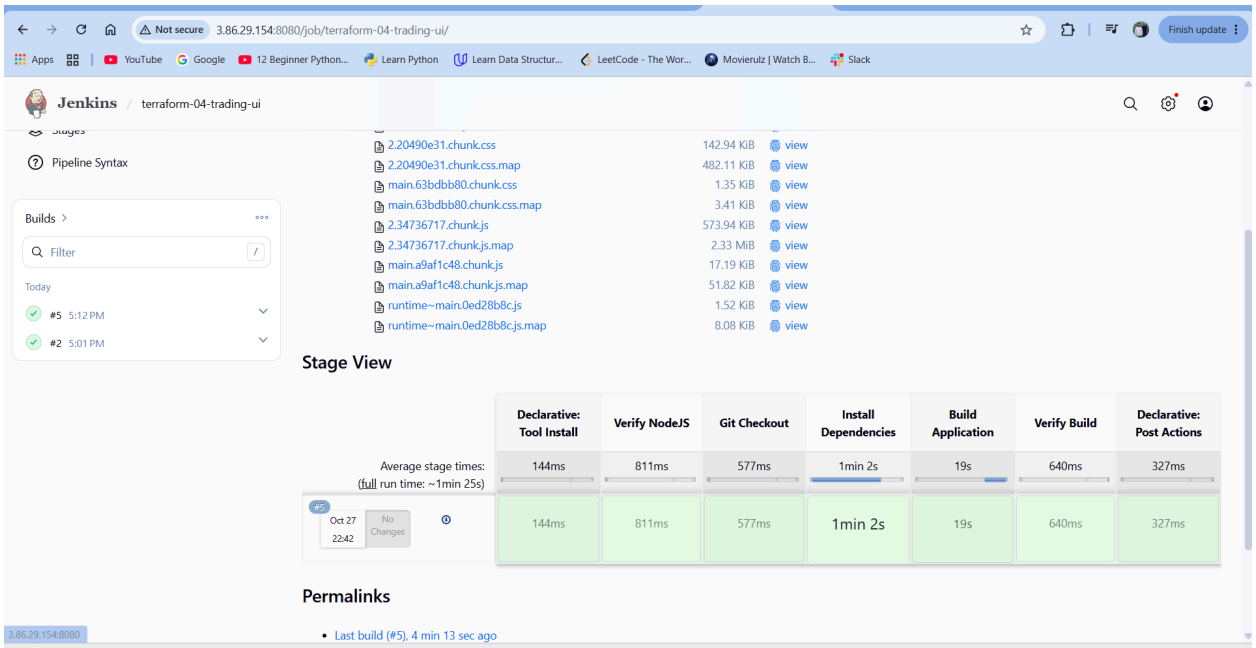
Issue 3: Unsupported Platform — fsevents

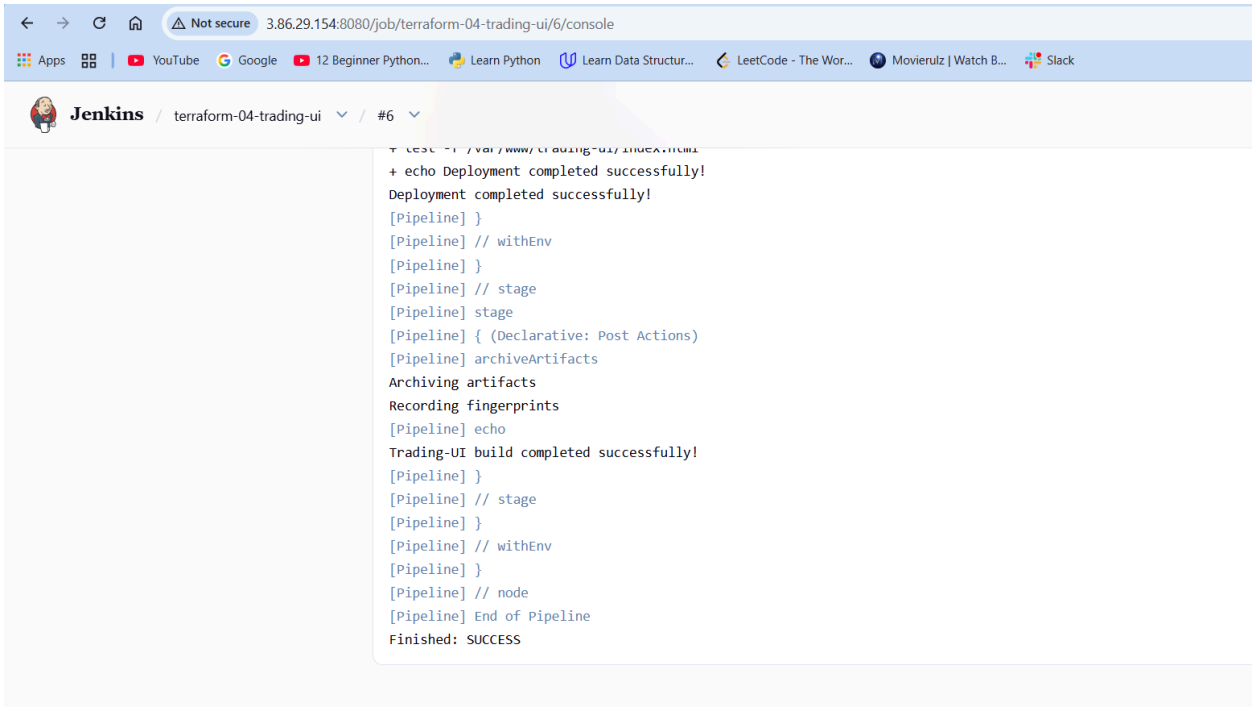
Error:

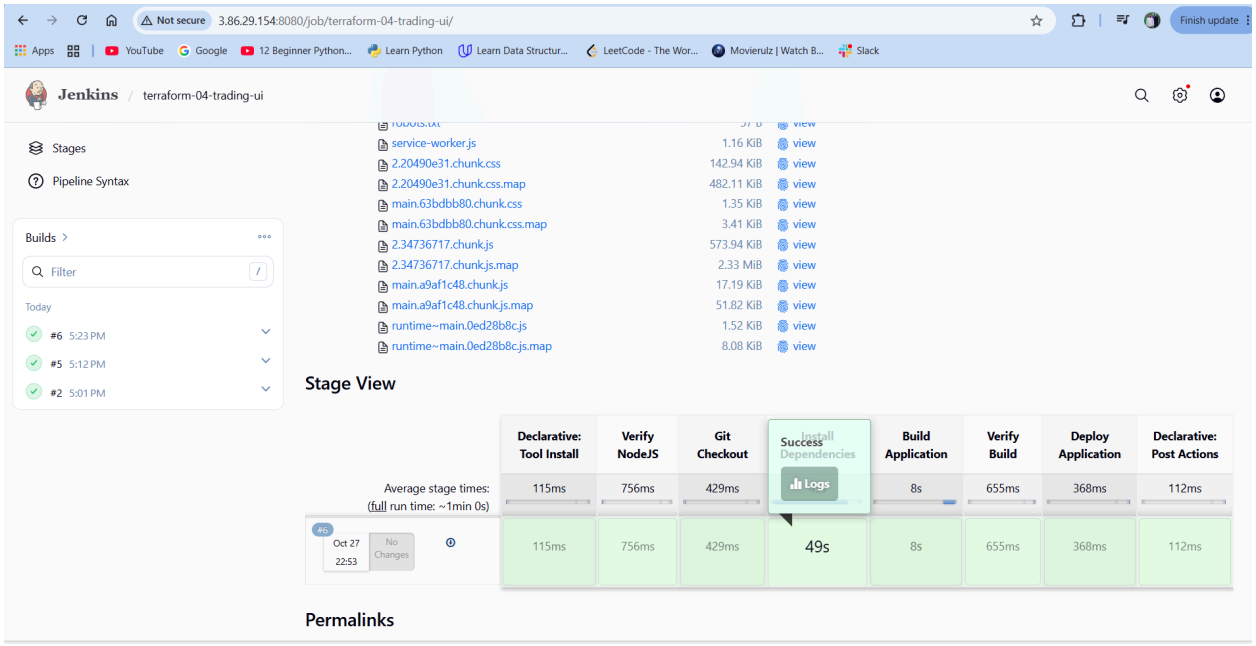
npm error code EBADPLATFORM
npm error notsup Unsupported platform for fsevents@1.2.9
npm error notsup wanted {"os":"darwin"}
npm error notsup current {"os":"linux"}

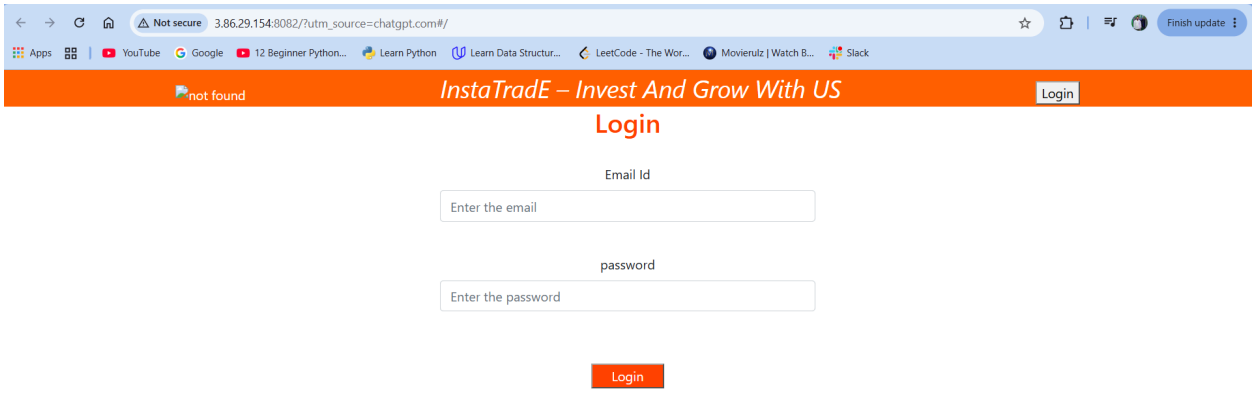
Cause: Outdated dependency lockfile containing macOS-specific packages.

Solution: For this lab, bypassed the old loc


Not secure 3.86.29.154:8080/job/terraform-04-trading-ui/
Jenkins / terraform-04-trading-ui
Pipeline Syntax
Builds
Filter
Today
#5 5:12 PM
#2 5:01 PM
2.20490e31.chunk.css 142.94 KiB view
2.20490e31.chunk.css.map 482.11 KiB view
main.63bdbb80.chunk.css 1.35 KiB view
main.63bdbb80.chunk.css.map 3.41 KiB view
2.34736717.chunk.js 573.94 KiB view
2.34736717.chunk.js.map 2.33 MiB view
main.a9af1c48.chunk.js 17.19 KiB view
main.a9af1c48.chunk.js.map 51.82 KiB view
runtime~main.0ed28b8c.js 1.52 KiB view
runtime~main.0ed28b8c.js.map 8.08 KiB view
Stage View
Declarative: Tool Install
Verify NodeJS
Git Checkout
Install Dependencies
Build Application
Verify Build
Declarative: Post Actions
Average stage times:
(full run time: ~1min 25s)
144ms 811ms 577ms 1min 2s 19s 640ms 327ms
#5 Oct 27 22:42 No Changes
144ms 811ms 577ms 1min 2s 19s 640ms 327ms
Permalinks
Last build (#5), 4 min 13 sec ago


Not secure 3.86.29.154:8080/job/terraform-04-trading-ui/6/console
Jenkins / terraform-04-trading-ui / #6
+ echo Deployment completed successfully!
Deployment completed successfully!
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] echo
Trading-UI build completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS





5). Explain **10 Maven commands**.

1. What is Maven?

Apache Maven is a build automation and dependency management tool mainly used for Java applications.

In real-time DevOps projects, Jenkins uses Maven to compile Java code, run tests, package applications as JAR/WAR files, and publish artifacts to repositories such as Nexus.

Developer pushes Java code to GitHub

Jenkins checks out source code

Maven compiles and tests application

Maven creates JAR / WAR artifact

Artifact uploaded to Nexus / deployed to server

2. Ten important Maven commands

No.	Command	Purpose
1	`mvn -version`	Check installed Maven and Java versions
2	`mvn validate`	Validate project configuration
3	`mvn compile`	Compile Java source code
4	`mvn test`	Execute unit tests
5	`mvn package`	Generate JAR or WAR file
6	`mvn verify`	Run lifecycle checks, including verification
7	`mvn install`	Install artifact into local Maven repository
8	`mvn clean`	Delete previous build output

9	`mvn dependency:tree`	Display project dependencies
10	`mvn deploy`	Publish artifact to remote repository, such as Nexus

Real-time example

Suppose your team is developing a Java banking application.

- Developer pushes code to GitHub.
- Jenkins runs `mvn clean package`.
- Maven compiles and tests the project.
- Maven creates `banking-app.war`.
- Jenkins publishes the WAR to Nexus or deploys it to Tomcat.

Important: Maven requires a Java/Maven project with a `pom.xml` file. Your previous Trading-UI React application uses npm instead of Maven.

